Glink

RENDER DEPLOYMENT GUIDE

Deploying Glink on Render

This guide puts Glink online on Render, step by step, mostly by clicking through the Render dashboard. It assumes the code is already on GitHub at `github.com/philippython/glink`. It takes about 20 minutes, plus any time spent setting up Stripe and email.
Render's screens change from time to time. If a button has a slightly different name, look for the nearest match.

1 What you'll end up with

2 Before you start

3 Create the service with the Blueprint (recommended)

4 Or create it by hand

5 Set your address and check it works

6 Your own domain (optional)

7 Payments, email and social sign-in

8 Updating

9 Backups

10 Logs and monitoring

11 A demo or staging copy

12 Troubleshooting

13 Launch checklist

Generated from docs/RENDER.md

1. What you'll end up with

One Render **Web Service** that runs Glink, serving both the website and the API, with a **persistent disk** attached for the database and uploaded videos. Render provides HTTPS and an `onrender.com` address automatically.

PIECE	SETTING
Plan	**Starter or higher.** Disks need a paid instance. The free plan has no disk and goes to sleep, so every restart would wipe all accounts and videos.
Instances	**Exactly 1.** The database and live updates live inside one process. Don't turn on autoscaling.
Disk	Mounted at `/var/data`, 10 GB to start (videos take ~50–100 MB each). You can grow it later but not shrink it.
Node	Version 24 (pinned by the `.node-version` file and `NODE_VERSION`)

Check render.com/pricing for current prices of the instance and disk.

Memory and uploads. Videos are held in memory while they upload. Starter has about 512 MB of RAM, so the Blueprint sets `MAX_UPLOAD_MB=150`. For longer, higher-quality videos, use the Standard plan (2 GB) and raise it to `500`.

2. Before you start

- [ ] A Render account (render.com). Sign up with GitHub, which makes connecting the repository easiest.
- [ ] The repository on GitHub. It includes `render.yaml` and `.node-version`, which this guide relies on.
- [ ] Optional now, or add later: Stripe keys (card payments), Resend keys (email), Google sign-in keys, a custom domain. See section 7.

3. Create the service with the Blueprint (recommended)

The repository contains `render.yaml`, a *Blueprint* that tells Render exactly what to create: the service, its disk, the build and start commands, the health check, and the settings.

1. In the Render dashboard, click **New** → **Blueprint**.
2. Connect GitHub if asked, and choose the **philippython/glink** repository and the `main` branch.

3. Render reads `render.yaml` and shows one web service called **glink** with a disk called **glink-data**. Before applying, you can change the **region**, which can't be moved later. Pick the one closest to your users.
4. Render asks for the values marked as secret. For now:
   - `APP_URL`: enter `https://glink.onrender.com`. You'll confirm the exact address in section 5.
   - Leave Stripe, Resend, Google and Anthropic **empty** for now; add them in section 7. Empty values are fine. Glink simply keeps those features switched off.
5. Click **Apply**. The first build takes a few minutes. Watch it in the service's **Events** / **Logs** tab.

When it finishes, the logs end with a line like `Glink API on http://localhost:10000`. Render routes your public address to that port automatically.

---

4. Or create it by hand

If you prefer not to use the Blueprint: **New** → **Web Service** → choose the repository, then fill in:

FIELD	VALUE
Runtime / Language	Node
Branch	`main`
Build command	`npm ci --include=dev && npm run build`
Start command	`npm start`
Instance type	Starter (or larger)
Health check path	`/api/health` (under *Advanced*)
Auto-deploy	On

Then, before the first deploy:

- **Disk** (*Advanced* → *Add disk*, or the **Disks** page later): name `glink-data`, mount path `/var/data`, size 10 GB.
- **Environment** variables:

KEY	VALUE
`NODE_ENV`	`production`
`NODE_VERSION`	`24`
`APP_URL`	`https://glink.onrender.com` (adjusted in section 5)
`DATABASE_PATH`	`/var/data/glink.db`
`UPLOAD_DIR`	`/var/data/uploads`

KEY	VALUE
`BACKUP_DIR`	`/var/data/backups`
`MAX_UPLOAD_MB`	`150` on Starter, `500` on Standard or larger

> **Why `--include=dev` in the build command?** With `NODE_ENV=production`, a plain `npm ci` skips development packages, including the tools that build the website (TypeScript and Vite), and the build fails with “vite: not found”. Including them for the build is safe; the running app doesn’t use them.

Don’t set `PORT`. Render provides it, and Glink reads it automatically.

---

5. Set your address and check it works

1. At the top of the service page, Render shows the public address, e.g. `https://glink.onrender.com`. If the name was taken, it adds a suffix like `glink-ab12.onrender.com`.
2. Go to **Environment** and make sure `APP_URL` is **exactly** that address: `https://`, no trailing slash. Save; Render redeploys.
3. Open `https://YOUR-ADDRESS/api/health`. You should see something like:

   ```
   {"ok":true,"ai":false,"providers":
   [],"demo":false,"testFunds":false,"payments":"none","email":false}
   ```

   `demo` and `testFunds` are `false` in production, which is correct. `payments` and `email` change once you add those keys.
4. Open the site, click **Sign up**, choose **I’m a brand**, and create your brand. You become its **Owner**.

The same checks were rehearsed with Render’s exact build and start commands, on a fresh copy of the repository: a clean production build, startup on port 10000, health check, sign-up, the database written to the disk folder, and `npm run backup`.

---

6. Your own domain (optional)

1. Service → **Settings** → **Custom Domains** → **Add**, e.g. `app.yourbrand.com`.
2. At your DNS provider, add the record Render shows. For a subdomain this is usually a `CNAME` pointing to your `onrender.com` address. Render issues the HTTPS certificate automatically once DNS is verified.
3. Change `APP_URL` to `https://app.yourbrand.com` and save.
4. Update anything that points at the old address: the Stripe webhook URL, Google/Facebook/Apple callback URLs, and the Resend domain if you send from this domain.

7. Payments, email and social sign-in

Add these on the service's **Environment** page. Each save redeploys the app. Full setup steps for each provider are in `docs/DEPLOYMENT.md` (sections 8 and 9). Below is what's specific to Render.

Stripe (card top-ups and creator payouts)

- `STRIPE_SECRET_KEY`: from Stripe → Developers → API keys. Use `sk_test_…` until you've tried everything.
- In Stripe → Developers → Webhooks, add the endpoint `https://YOUR-ADDRESS/api/stripe/webhook` with the events `checkout.session.completed`, `checkout.session.async_payment_succeeded` and `account.updated`. Include connected-account events. Copy its signing secret into `STRIPE_WEBHOOK_SECRET`.
- `STRIPE_CURRENCY` defaults to `usd`.
- Check: `/api/health` shows `"payments":"stripe"`. As a brand owner, Billing → *Add funds* → pay with test card `4242 4242 4242 4242`; the wallet updates within seconds. Stripe's webhook page should show the delivery with a `200` response.

Resend (password-reset and team-invite emails)

- `RESEND_API_KEY`, plus `EMAIL_FROM`, e.g. `Glink <hello@yourbrand.com>`, on a domain verified in Resend.
- Check: `/api/health` shows `"email":true`. Request a password reset and the email arrives.

Google sign-in

- `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`. In Google Cloud, the authorised redirect URI is `https://YOUR-ADDRESS/api/auth/oauth/google/callback`.
- Facebook (`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`) and Apple (`APPLE_*`) work the same way. Add the keys in the dashboard; they aren't in the Blueprint.

AI checks (optional): `ANTHROPIC_API_KEY`.

8. Updating

With auto-deploy on, **every push to `main` deploys automatically**:

```
git add -A
git commit -m "Describe the change"
git push
```

- Database changes are applied automatically when the new version starts.
- A service with a disk can't run old and new versions side by side, so each deploy has a short pause (usually under a minute) while the old one stops and the new one starts. Open pages reconnect on their own.
- Before a big update, take a backup (section 9).

- To undo a bad deploy: service → **Events** → find the previous successful deploy → **Rollback**. A rollback restores the code, not the data, so for data problems restore a backup.

---

9. Backups

What matters is on the disk: `glink.db` (all accounts, campaigns and money records) and `uploads/` (videos).

- **Render disk snapshots.** Render takes automatic snapshots of disks. You can see them, and restore one, from the service's **Disks** page. Check there for the schedule and how long they're kept on your plan.
- **Your own copies.** Open the service's **Shell** tab and run:

  ```
  npm run backup
  ```

  It writes a consistent copy to `/var/data/backups/` while the app keeps running, and keeps the newest 14. Run it before every big update.
- **Off Render.** Snapshots and `/var/data/backups` live with the service. For safety against account-level mistakes, download a copy now and then. Render's SSH access lets you `scp` files from `/var/data/backups`; see Render's SSH docs.

Restoring a backup copy: in the Shell, stop writing traffic first, by suspending the service or doing it at a quiet time. Then:

```
cp /var/data/backups/glink-<date>.db /var/data/glink.db
rm -f /var/data/glink.db-wal /var/data/glink.db-shm
```

Then **Manual Deploy** → *Restart service*.

---

10. Logs and monitoring

- **Logs** tab: startup lines, errors, and any email that couldn't be sent. `ExperimentalWarning: SQLite is an experimental feature` is expected and harmless.
- **Health check:** Render calls `/api/health`. If it stops answering, Render marks the deploy failed or restarts the service.
- **Metrics** tab: watch memory. If it climbs near the plan's limit during uploads, lower `MAX_UPLOAD_MB` or move to a bigger plan.
- **Disk usage** shows on the **Disks** page. Increase the size before it fills; videos add up.
- **Notifications:** in account settings, turn on email or Slack alerts for failed deploys.

11. A demo or staging copy

To show Glink to people without touching real data, create a **second** service the same way: for example, apply the Blueprint again with a different service name, or duplicate the service. Give it its own disk, and set:

- `SEED_DEMO=true`, which creates the demo brand and creators (password `demo1234`) on its empty database;
- `ALLOW_TEST_FUNDS=true`, which turns on the “Add test funds” button;
- Stripe **test** keys only, if any.

Never set `SEED_DEMO=true` or `ALLOW_TEST_FUNDS=true` on the service real customers use.

Quick throwaway demo on the free plan: possible without a disk, using the default database location, but all data is wiped whenever the service restarts or redeploys, and it sleeps when idle. Fine for a five-minute look, not for anything you want to keep.

12. Troubleshooting

WHAT YOU SEE	FIX
Build fails: `vite: not found` or `tsc: not found`	The build command must be `npm ci --include=dev && npm run build` (section 4).
Start fails: `No such built-in module: node:sqlite`	Node is too old. Set `NODE_VERSION=24` and make sure `.node-version` is in the repo.
Start fails: `Cannot find package 'tsx'`	Dependencies didn’t install. Check the build log for an `npm ci` error, then *Clear build cache & deploy*.
Accounts disappear after a deploy	The disk isn’t attached, or the paths don’t match. Check the disk is mounted at `/var/data` and `DATABASE_PATH=/var/data/glink.db`.
“Cross-site request blocked” on sign-up or log-in	`APP_URL` doesn’t exactly match the address in the browser: `https://`, no trailing slash, the custom domain if you use one.
Can’t stay logged in	`APP_URL` must start with `https://`.
Deploy stuck or marked failed	Open the deploy’s logs. If the app started, check the health check path is `/api/health`.
Uploads fail or the service restarts during uploads	Out of memory. Lower `MAX_UPLOAD_MB` or move to a bigger plan (section 1).
Card payment went through but the wallet didn’t change	The Stripe webhook URL, events or `STRIPE_WEBHOOK_SECRET` are wrong. Stripe’s webhook page shows each delivery and Glink’s response.

WHAT YOU SEE	FIX
Google says `redirect_uri_mismatch`	The redirect URI in Google must be exactly `APP_URL` + `/api/auth/oauth/google/callback`.
Password-reset emails don't arrive	`/api/health` should say `"email":true`, and `EMAIL_FROM` must use a domain verified in Resend. Errors appear in the Logs.

13. Launch checklist

- [ ] Paid instance, **one** instance, disk mounted at `/var/data`
- [ ] `APP_URL` is the exact `https://` address (custom domain if you have one)
- [ ] `/api/health` shows `"demo":false` and `"testFunds":false`
- [ ] Stripe **live** keys and live webhook set; one real top-up and one creator payout tried
- [ ] Resend set up and a password-reset email received
- [ ] Social sign-in callback URLs use the final address
- [ ] First backup taken with `npm run backup`; you know where disk snapshots are
- [ ] Deploy-failure notifications turned on